

BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent`.
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
   Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

If the environment has many possible states, and the agent operates for a long time, the table becomes impossibly large.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The Condition (IF): "What the world is like now" based on incoming sensor data.
In the code, this maps directly to evaluating the current state keys, such as if percept['food_ahead']: , elif percept['wall_ahead']:, elif percept['opponent_ahead']:, elif percept['toxin_ahead']:

The Action (THEN): "What action I should do now," which then flows directly to the actuators.
In the code, this maps to the immediate commands sent back to the game, like return 'Forward', return 'TurnLeft' or return 'TurnRight'.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

In partially observable environments, a simple reflex agent often falls into unavoidable infinite loops.

The grid agent that only has a "FRONT" sensor, but no "LOCATION" sensor.
If it walks into a corner and sees a wall ahead, the rule says "TurnLeft". If it turns and immediately faces an adjacent boundary wall, it turns left again and moves "Forward" into the same small space, bouncing back to the original wall and triggering the exact same "TurnLeft" rule again. Forever.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the

"Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Transition Model: The agent's internal world model distinguishes between static and dynamic elements of the environment. Walls and toxins are treated as permanent. Once sensed, their location is stored in self.obstacles and never removed, since these hazards don't move. Opponents, by contrast, are excluded from self.obstacles entirely, since the environment's actual transition rules allow them to move to a new cell on every step.

Sensor Model: update_state() method translates the agent's own actions into a predicted change in position and orientation, rather than sensing them directly. Based on self.last_action, a 'Forward' command shifts self.current_pos by one cell along the axis matching self.facing, while 'TurnLeft'/'TurnRight' rotate self.facing through the direction list - giving the agent a running estimate of where it is and which way it's pointing, purely from memory of its own actions.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING